

Transformer limitations

Grandes Modelos de Linguagem

Karolina Azevedo
Igor Viveiros

Marcus Vinicius Moraes Oliveira
Luis Antonio Duarte Sousa



12 de novembro de 2025

Introdução

Limitações dos Transformers e Mitigações

On Limitations of the Transformer Architecture

Representational Strengths and Limitations of Transformers

Theoretical Limitations of Self-Attention in Neural Sequence Models

Transformers Learn Shortcuts To Automata

Conclusão

Desde o trabalho *Attention is All You Need* (Vaswani et al., 2017), Transformers se tornaram a base de LLMs como BERT e GPT, dominando tarefas de linguagem, visão e multimodalidade.

Apesar do sucesso prático, há questões teóricas fundamentais:

- ▶ Até que ponto conseguem lidar com composições de funções, raciocínio, memória e estruturas hierárquicas?
- ▶ Por que modelos tão grandes ainda cometem “alucinações” lógicas?

Estudar essas limitações permite compreender:

- ▶ Por que erros sistemáticos ocorrem;
- ▶ Quais tarefas estão fora do alcance teórico da arquitetura;

Autores: Peng, Narayanan & Papadimitriou (2024).

Ideia Central: O artigo investiga limitações formais da arquitetura Transformer que podem explicar, em parte, por que modelos de linguagem "alucinam".

Problema: Dada a descrição de duas funções $g:A \rightarrow B$ e $f:B \rightarrow C$ na forma de sentenças no prompt, e uma consulta do tipo "qual é $f(g(x))$?", pode uma camada de atenção com parâmetros limitados responder corretamente para todo par de funções? Chamam isso de problema de função-composição.

On Limitations of the Transformer Architecture II

Intuição:

Suponha que queremos responder: *"Se João é pai de Maria e Maria é mãe de Ana, quem é o avô de Ana?"*

Resolver isso exige duas aplicações de uma função relacional:

$$g(x) = \text{mãe}(x), \quad f(y) = \text{pai}(y),$$

e o modelo deve calcular $f(g(\text{Ana}))$.

Em um Transformer, o token “Ana” precisa recuperar informação de “Maria” e, indiretamente, de “João”. Mas a camada de atenção faz uma média ponderada (softmax) sobre todos os tokens. Essa operação suaviza as relações (perda de informação) e não consegue preservar exatamente a sequência de duas aplicações de função quando o número de possíveis entidades ou relações (n) é muito grande.

Consequência: o modelo tende a gerar respostas plausíveis, mas frequentemente incorretas — uma forma teórica de “alucinação”.

On Limitations of the Transformer Architecture III

Causa:

Vamos pensar na camada de atenção do Transformer como uma rede de comunicação entre diferentes partes do texto. Cada cabeça de atenção é como um canal que pode carregar uma certa quantidade de informação. Ela transmite apenas vetores de dimensão d , e cada número dentro desses vetores possui uma precisão numérica de p bits. Ou seja, uma cabeça tem uma *largura de banda* finita, consegue passar no máximo $d \times p$ bits de informação útil.

Se o modelo tem H cabeças, a quantidade total de informação que pode circular em uma camada é aproximadamente $H(d + 1)p$ bits.

Agora, para uma tarefa de composição, o modelo precisa combinar as informações de duas funções f e g , que estão espalhadas em partes diferentes do prompt. Só que, para distinguir todas as possíveis combinações de funções sobre um domínio de tamanho n , seria preciso transmitir cerca de $n \log n$ bits de informação. ($\log_2(n^n)$)

Portanto, se a capacidade total da camada $H(d + 1)p$ for menor que essa necessidade $n \log n$, a camada simplesmente não consegue carregar todos os detalhes da função f que seriam necessários para compor corretamente.

On Limitations of the Transformer Architecture IV

Chain-of-Thought (CoT):

O raciocínio em cadeia (*chain-of-thought*) pode ajudar o modelo a dividir um problema complexo em pequenos passos intermediários — como fazer subcálculos antes de chegar à resposta final. Porém, há um custo: cada passo do CoT consome espaço no prompt e mais operações internas, representando uma função à mais na cadeia de raciocínio, isto é: $f_n(f_{n-1}(\dots f_1(x)))$.

Memória:

Muitos raciocínios compostos exigem lembrar resultados intermediários ou explorar múltiplas possibilidades. Isso equivale a ter uma “memória de trabalho” proporcional ao tamanho do problema ou a um tipo de não-determinismo.

- ▶ A operação de atenção (*softmax*) age como um funil de informação, limitando o quanto uma camada pode combinar dados distintos, isso restringe a capacidade de compor funções complexas.
- ▶ O uso de *chain-of-thought* ajuda a contornar essa limitação, mas o número de passos cresce rapidamente.
- ▶ No todo, Transformers operam com muito uso de memória que acaba sendo um limitante.

Autores: Clayton Sanford, Daniel Hsu & Matus Telgarsky (2023).

Ideia Central: O artigo identifica tarefas primitivas que explicam quando a autoatenção é especialmente eficiente e quando ela é intrinsecamente limitada. Mostra que a dimensão de embedding m , o número de cabeças e a profundidade controlam fortemente o poder representacional.

Problema: Que funções sequência-para-sequência podem ser representadas de forma eficiente por unidades de self-attention de dimensão limitada? Os autores formalizam três protótipos:

- ▶ qSA (q-sparse averaging): cada posição devolve a média de q entradas indicadas por um índice; usada para medir a dependência de m com q .
- ▶ Match2: detectar, para cada posição, se existe um par que satisfaça uma relação simples (problema intrinsecamente pareado).
- ▶ Match3: detectar triplas que satisfazem uma condição (problema triple-wise).

Strengths: Análise do problema qSparse Averaging (*qSA*)

Definição: Dado um conjunto de entradas

$$X = ((y_1, z_1), (y_2, z_2), \dots, (y_N, z_N)),$$

onde:

- ▶ $z_i \in \mathbb{R}^{d'}$ é o **vetor de dados** (valor);
- ▶ $y_i \subseteq [N]$ é um **subconjunto de índices** com $|y_i| = q$.

A função q -sparse averaging é definida como:

$$\text{qSA}(X)_i = \frac{1}{q} \sum_{j \in y_i} z_j$$

Cada saída é a média dos q vetores z_j cujos índices estão em y_i .

Exemplo simples:

Considere $N = 4$, $q = 2$ e $d' = 1$.

$$z = (1, 3, 5, 7) \quad y = (\{1, 2\}, \{2, 3\}, \{3, 4\}, \{1, 4\})$$

$$\text{qSA}(X)_1 = \frac{1}{2}(z_1 + z_2) = \frac{1}{2}(1 + 3) = 2$$

$$\text{qSA}(X)_2 = \frac{1}{2}(z_2 + z_3) = \frac{1}{2}(3 + 5) = 4$$

$$\text{qSA}(X)_3 = \frac{1}{2}(z_3 + z_4) = \frac{1}{2}(5 + 7) = 6$$

$$\text{qSA}(X)_4 = \frac{1}{2}(z_1 + z_4) = \frac{1}{2}(1 + 7) = 4$$

$$\Rightarrow \text{qSA}(X) = (2, 4, 6, 4)$$

Intuição: Cada posição i “olha” para um subconjunto y_i de outros tokens e calcula a média de seus valores. Essa operação é análoga ao que o **mecanismo de atenção** faz — agregando informações de tokens relevantes.

Transformers (Eficiência Logarítmica): Uma única unidade de autoatenção aproxima qSA com um tamanho de modelo que escala apenas logaritmicamente com o comprimento da sequência, isto é $q \log(N)$, tornando-o altamente eficiente para agregações esparsas e de longo alcance.

MLPs (Crescimento Polinomial): As Redes Totalmente Conectadas (MLPs) são fundamentalmente inadequadas, exigindo uma largura de camada oculta que escala polinomialmente ($\Omega(N^d)$) com o comprimento da sequência, tornando-se computacionalmente intratáveis.

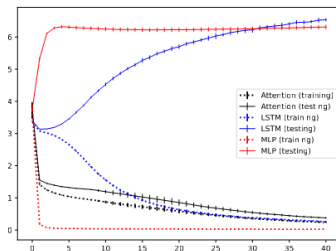
Recorrentes (RNNs): são limitadas por um gargalo de memória, exigindo um estado oculto que escala linearmente ($\Omega(N)$ bits) com o comprimento da sequência para resolver a tarefa.

O Papel Crítico da dimensão do embedding de atenção, m

Suficiência: O sucesso na tarefa qSA depende de a dimensão do embedding (m) ser suficientemente grande (Teorema 2: $m \gtrsim q$) em relação ao parâmetro de esparsidade (q).

Necessidade: Se a dimensão do embedding é muito pequena (Teorema 4: $mp \leq cq$), c constante, p precisão em bits, a tarefa se torna impossível para um Transformer de camada única, estabelecendo m como um recurso representacional fundamental.

Implicação Prática: A dimensão do embedding (m) não é apenas um hiperparâmetro, mas deve ser dimensionada em proporção à complexidade das interações esparsas que o modelo deve aprender.



Tarefa *Match2*: definição, exemplo e eficiência do Transformer

Definição:

$$\text{Match2}(X)_i = \begin{cases} 1, & \text{se } \exists j \text{ tal que } x_i + x_j \equiv 0 \pmod{M}, \\ 0, & \text{caso contrário.} \end{cases}$$

Exemplo: Considere $N = 4$ e módulo $M = 5$.

$$X = (x_1, x_2, x_3, x_4) = (1, 4, 2, 3)$$

- ▶ $i = 1$, $x_1 + x_2 = 1 + 4 = 5 \equiv 0 \pmod{5} \Rightarrow x_1$ e x_2 formam um par válido.
- ▶ $i = 3$, $x_3 + x_4 = 2 + 3 = 5 \equiv 0 \pmod{5} \Rightarrow x_3$ e x_4 também formam um par válido.

$$\Rightarrow \text{Match2}(X) = (1, 1, 1, 1)$$

Por que o Transformer é eficiente:

- ▶ O mecanismo de atenção calcula similaridades **par-a-par** entre tokens:

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^{\top})V$$

- ▶ A operação $QK^{\top}$ já realiza todas as interações bilineares (i, j) .
- ▶ Portanto, **Match2** é uma tarefa naturalmente **pairwise** — exatamente o tipo de dependência que a autoatenção modela.
- ▶ O Transformer pode representar *Match2* com uma única cabeça e embedding pequeno ($m = 3$) de forma eficiente.

Conclusão: O Transformer é ideal para tarefas **de segunda ordem** (pares), onde cada saída depende de relações diretas entre dois tokens.

Limitations: Por que o Transformer não é eficiente para resolver o *Match3(X)*

Teorema 7

- ▶ Para embedding m , número de cabeças H e precisão p fixos, existe um N suficientemente grande tal que nenhum transformer, $f(X)$, pode computar exatamente a função $\text{Match3}(X)$.
- ▶ Formalmente: se $mpH \leq \frac{cN}{\log \log N}$, então $f(X) \neq \text{Match3}(X)$ para alguns $X \in [M]^N$.

Definição da tarefa:

$$\text{Match3}(X)_i = 1_{\{\exists j_1, j_2 \text{ tal que } x_i + x_{j_1} + x_{j_2} = 0 \pmod{M}\}}$$

Exemplo: Considere $N = 4$ e módulo $M = 5$.

$$X = (x_1, x_2, x_3, x_4) = (1, 2, 3, 4)$$

Verificação:

- ▶ Para $i = 1$: $x_1 + x_2 + x_3 = 1 + 2 + 3 = 6 \equiv 1 \pmod{5}$
- ▶ Para $i = 2$: $x_2 + x_3 + x_4 = 2 + 3 + 4 = 9 \equiv 4 \pmod{5}$
- ▶ Para $i = 3$: $x_3 + x_1 + x_4 = 3 + 1 + 4 = 8 \equiv 3 \pmod{5}$
- ▶ Para $i = 4$: $x_4 + x_1 + x_2 = 4 + 1 + 2 = 7 \equiv 2 \pmod{5}$

Nenhuma tripla soma 0 módulo 5:

$$\Rightarrow \text{Match3}(X) = (0, 0, 0, 0)$$

Intuição do resultado:

- ▶ A autoatenção padrão realiza apenas **interações bilineares**:

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^T)V$$

Cada cabeça combina pares (i, j) — é uma operação **de segunda ordem**.

- ▶ A tarefa *Match3* requer **interações de terceira ordem** entre triplas (i, j_1, j_2) , isto é, dependências *triple-wise*.
- ▶ Com m e H fixos, a atenção não possui *largura de comunicação* suficiente para codificar todas as combinações triplas ($O(N^2)$ pares $\rightarrow O(N^3)$ triplas).
- ▶ Portanto, para representar Match3 é necessário que m , H ou a profundidade ($\#$ camadas de atenção) cresçam polinomialmente em N .

Conclusão: Transformers são eficientes apenas para tarefas **pairwise** (como *Match2*), tarefas **triple-wise** exigem *atenção de ordem superior* ou aumento polinomial de recursos.

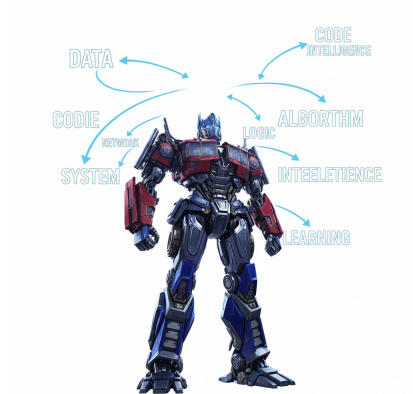
Priorize Agregação Flexível e Longa Distância: Transformers são a escolha ideal para tarefas que exigem agregação não-local e esparsa de informações, superando RNNs e MLPs nestes cenários.

Dimensione o Embedding Pela Complexidade: A dimensão do embedding (m) deve ser tratada como um recurso essencial diretamente ligado à complexidade da interação (q). Certifique-se de que m seja grande o suficiente para suportar as interações que o modelo deve aprender.

Use a Estrutura para Superar Limitações: Em tarefas de ordem superior ($3+$), a chave não é forçar o cálculo direto, mas projetar o modelo para alavancar pistas de localidade e ordem sequencial para decompor o problema em operações pairwise tratáveis.

Theoretical Limitations of Self-Attention in Neural Sequence Models I

- ▶ **Autor:** Michael Hahn (Stanford, 2020).
- ▶ **Ideia central:** Os Transformers são altamente eficazes, mas apresentam limitações teóricas de expressividade.
- ▶ **Problema:** São teoricamente capazes de modelar estruturas hierárquicas envolvendo recursão ilimitada?
- ▶ **Limites:** Especialmente quando uma linguagem é muito hierárquica.
- ▶ **Demonstração:** Formal mostrando matematicamente o que o self-attention é capaz de representar.



O que o artigo testa:

O autor avalia se Transformers conseguem representar estruturas hierárquicas, testando duas linguagens formais:

- ▶ **PARITY** – verificar se há número par de 1s.

$$L = \{x \in \{0, 1\}^* \mid \text{a contagem de 1s em } x \text{ é par}\}$$

- Transformers fixos não mantêm alternância por muito tempo.
- Conforme a sequência cresce, perdem noção de periodicidade.
- Não lida bem com PARITY, falha em quase todas as linguagens regulares que dependem de contagem.

- ▶ **2DYCK** – verificar se parênteses estão balanceados.

$$L = \{(), (()), ()[], \dots\}$$

- Mais complexa.
- Para processar é preciso memória tipo pilha.
- Dificuldade em reconhecer padrões muito aninhados.
- Não lidam bem com linguagens hierárquicas.

Essas linguagens representam respectivamente a capacidade de contagem e de reconhecer hierarquias (recursão).

O que ele descobriu? Não consegue resolver essas tarefas perfeitamente.

Porque? Não tem memória infinita, não lê passo a passo. Olha tudo ao mesmo tempo, perde a noção de ordem e profundidade.

O que isso quer dizer? Self-attention não pode reconhecer linguagens não lineares, como PARITY e 2DYCK, a menos que o número de camadas, cabeças ou parâmetros cresça com o comprimento da entrada. Ele não pode simular pilha.

Mas se tem limites porque Transformers funcionam tão bem? Mundo real não é infinito, frases curtas, aproximam bem o que precisamos.



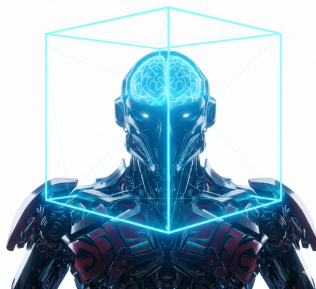
Comparação com outras arquiteturas:

Tabela 1: Comparação de Modelos para Reconhecimento da Linguagem de Dyck

Modelo	Tipo de Memória	Capaz de reconhecer 2DYCK?
RNN simples	Sequencial	✗
LSTM	Pilha implícita	✓
Transformer	Atenção global, sem pilha	✗ (teoricamente)

Síntese do estudo

- ▶ Apesar das limitações teóricas, Transformers têm desempenho prático excelente.
- ▶ O self-attention não entende bem hierarquias profundas.
- ▶ Isso explica porque podem errar em estruturas complexas.
- ▶ Mesmo assim funcionam bem para maioria dos textos humanos.



- ▶ Autores: Liu, Ash, Goel, Krishnamurthy Zhang
- ▶ Como os transformers realizam o raciocínio algorítmico
- ▶ O paper investiga o uso da arquitetura transformer para diferentes problemas de simulação de um semiautomato

O que o artigo testa:

Como Transformers simulam autômatos sem recorrência?

- ▶ Autômatos são sistemas que mudam de estado a cada símbolo de entrada. Normalmente, isso exige processamento sequencial (recorrente).
- ▶ Um **semiautômato** é definido como um triplo:

$$A = (Q, \Sigma, \delta)$$

onde:

- Q é o conjunto de **estados**;
- Σ é o **alfabeto de entrada**;
- $\delta : Q \times \Sigma \rightarrow Q$ é a **função de transição**.

Em cada passo de tempo t , o autômato atualiza seu estado segundo:

$$q_t = \delta(q_{t-1}, \sigma_t), \quad t = 1, \dots, T$$

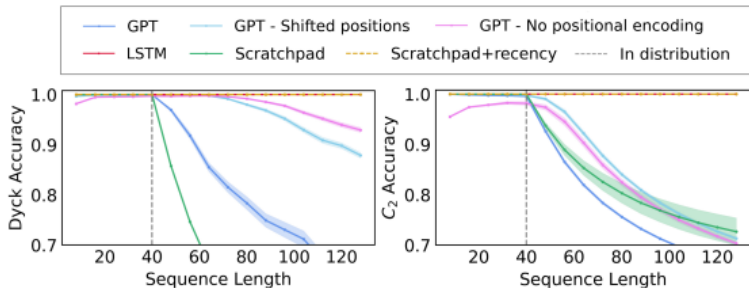
RNN x Transformers::

- ▶ RNNs naturalmente simulam autômatos, pois atualizam o estado a cada passo.
- ▶ Transformers não são recorrentes, mas podem processar toda a sequência em paralelo.
 - **RNN:** $\mathcal{O}(T)$
 - **Transformer:** $\mathcal{O}(L)$ — processa toda a sequência em **paralelo**, usando L camadas de autoatenção.

Resultados principais:

- ▶ Transformers podem simular autômatos com profundidade muito menor que T , usando paralelismo. $O(\log T)$ ou $O(1)$
- ▶ treinando com SGD, Transformers aprenderam todos os autômatos com alta precisão > 99
- ▶ Porém existe alta variância entre as execuções e o treinamento do modelo é instável
- ▶ Além disso com testes com T diferente do treino o modelo transformer falha severamente comparado com LSTMs

Resultados principais:



Síntese do estudo

- ▶ Transformers conseguem simular automatons com alta precisão
- ▶ Transformers não conseguem fazer a "lógica" e o passo a passo algorítmico, apenas conseguem achar padrões estatísticos
- ▶ A solução com Transformers não é robusta para um domínio fora dos dados de treino, porém com chain-of-thoughts o modelo se torna um pouco mais robusto

Lição Principal:

Os Transformers são bons mas também são limitados.

- ▶ **Reflexão:** Escalar indefinidamente é aplicar força bruta computacional a um problema essencialmente lógico.
- ▶ **Direção Futura:** O progresso real virá de arquiteturas mais inteligentes, não apenas maiores.

Futuro da IA: Inteligência emergirá de *estruturas novas*, não do tamanho.

- [1] B. Liu, J. T. Ash, S. Goel, A. Krishnamurthy e C. Zhang, “Transformers Learn Shortcuts to Automata,” em *Proceedings of ICLR 2023*, arXiv preprint, also at OpenReview, 2023. endereço: <https://openreview.net/forum?id=De4FYqjFueZ>
- [2] C. Sanford, D. J. Hsu e M. J. Telgarsky, “Representational Strengths and Limitations of Transformers,” em *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. endereço: https://proceedings.neurips.cc/paper_files/paper/2023/hash/73bf692447f174984f30499ec9b20e04-Abstract-Conference.html
- [3] M. Hahn, “Theoretical Limitations of Self-Attention in Neural Sequence Models,” *Transactions of the Association for Computational Linguistics*, v. 8, pp. 156–171, 2020. DOI: [10.1162/tacl_a_00306](https://aclanthology.org/2020.tacl-1.11/) endereço: <https://aclanthology.org/2020.tacl-1.11/>
- [4] B. Peng, S. Narayanan e C. Papadimitriou, “On Limitations of the Transformer Architecture,” *arXiv preprint arXiv:2402.08164v2*, 2024. endereço: <https://arxiv.org/html/2402.08164v2>